Promesas, Async/Await y Solicitudes Axios en Node.js y Express

Tiempo estimado necesario: **20 minutos**,

Objetivos

En esta lectura, aprenderás sobre:

- **Promesas**: El bloque de construcción fundamental para manejar operaciones asíncronas en JavaScript
- **Async/Await**: Una sintaxis moderna y más legible para trabajar con Promesas
- **Axios**: Un cliente HTTP ampliamente utilizado para realizar solicitudes de red

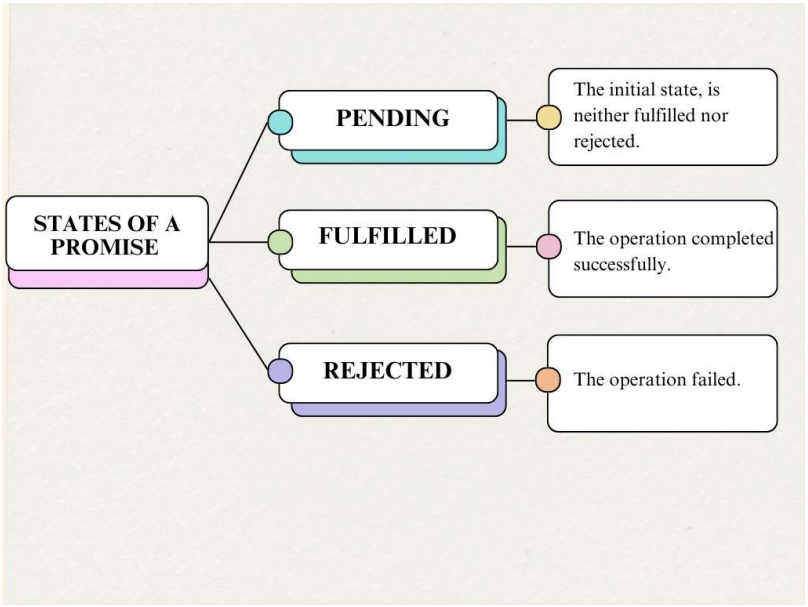
Introducción

El modelo de programación asincrónica de JavaScript es fundamental para desarrollar aplicaciones web receptivas y de alto rendimiento. En Node.js y Express, manejar operaciones asincrónicas de manera efectiva es esencial para tareas como realizar solicitudes a APIs, leer archivos o consultar bases de datos.

1. Promesas

Una Promesa es un objeto que representa la eventual finalización (o fallo) de una operación asíncrona y su valor resultante. Te permite encadenar operaciones de una manera más legible y manejable.

1.1 Estados de una Promesa



1.2 Creando una Promesa

Para crear una Promesa, utilizas el nuevo constructor Promise, que toma una función con dos parámetros: resolve y reject. La función resolve se llama cuando la operación asíncrona se completa con éxito, y la función reject se llama cuando falla.

```
// Creating a new Promise object and assigning it to the variable myPromise
const myPromise = new Promise((resolve, reject) => {
  // Simulating a condition with a boolean variable 'success'
  let success = true;
  // If the condition is true, call resolve to mark the promise as fulfilled
  if (success) {
    resolve("The operation was successful!");
  } else {
    // If the condition is false, call reject to mark the promise as rejected
    reject("The operation failed!");
  }
});
```

1.3 Uso de Promesas con .then() y .catch()

Puedes manejar el valor resuelto o el error utilizando los métodos `.then()` y `.catch()`. Estos métodos también devuelven Promesas, lo que te permite encadenar múltiples operaciones asincrónicas en secuencia.

```
// Execute the promise and handle the fulfilled and rejected states
myPromise
```

```
// Handle the resolved state of the promise
.then((message) => {
  // This block will execute if the promise is resolved
  console.log(message); // "The operation was successful!"
})
// Handle the rejected state of the promise
.catch((error) => {
  // This block will execute if the promise is rejected
  console.error(error); // "The operation failed!"
});
```

1.4 Ejemplo: Leer un archivo

Aquí hay un ejemplo utilizando el módulo fs.promises para leer un archivo. El módulo fs.promises proporciona métodos basados en Promesas para operaciones del sistema de archivos.

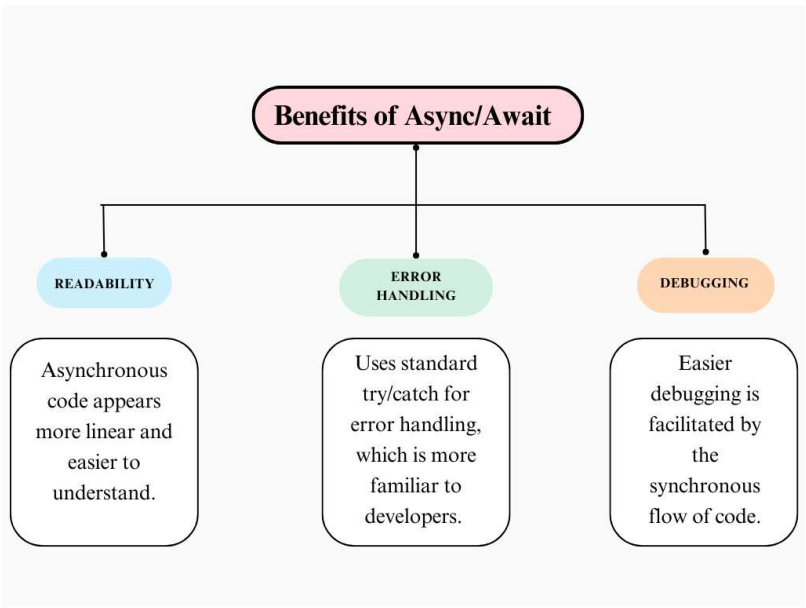
```
// Import the 'fs' module and use its promise-based methods
const fs = require('fs').promises;
// Read the content of the file 'example.txt' with 'utf8' encoding
fs.readFile('example.txt', 'utf8')
  // Handle the resolved state of the promise
  .then((data) => {
    // This block will execute if the file is read successfully
    console.log(data); // Print the file content to the console
  })
  // Handle the rejected state of the promise
  .catch((err) => {
    // This block will execute if there is an error reading the file
    console.error('Error reading file:', err); // Print the error message to the console
  });
```

2. Async/Await

Como quizás ya hayas aprendido, Java Script es un lenguaje de scripting de un solo hilo. Esto significa que el proceso solo puede ocurrir de manera secuencial y no se pueden llevar a cabo dos procesos simultáneamente. Esto es un gran obstáculo para cualquier lenguaje y JS lo resolvió introduciendo la programación asíncrona a través de Promesas. Si bien las Promesas resolvieron los problemas de la programación síncrona, los `then` anidados pueden complicar la estructura y la legibilidad del código.

Async y Await son azúcares sintácticos sobre Promesas, haciendo que el código asíncrono se parezca más al código síncrono, lo que es más fácil de leer y escribir. Una función async devuelve una Promesa, y puedes usar await dentro de una función async para pausar la ejecución hasta que una Promesa sea resuelta o rechazada.

2.1 Beneficios de Async/Await:



2.2 Usando async y await

Una función async siempre devuelve una Promise. Dentro de la función async, puedes usar la palabra clave await para pausar la ejecución hasta que una Promise sea resuelta o rechazada.

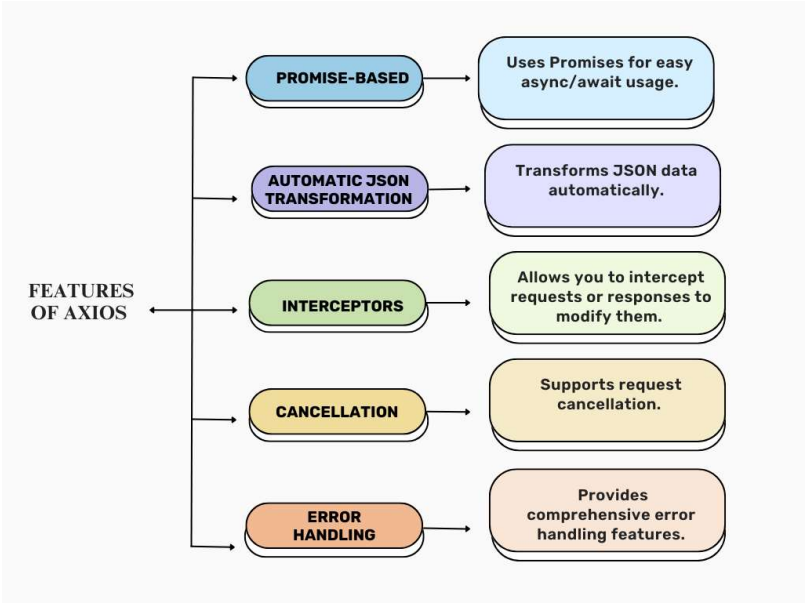
```
// Async function that wraps the operation
async function myAsyncFunction() {
```

```
  // Simulating a condition with a boolean variable 'success'
  let success = true;
  // If the condition is true, resolve with a success message
  if (success) {
    return "The operation was successful!";
  } else {
    // If the condition is false, throw an error to simulate rejection
    throw new Error("The operation failed!");
  }
}
// Using async function to handle Promise
async function executeAsyncFunction() {
  try {
    // Await the async function call to get the result
    const result = await myAsyncFunction();
    console.log(result); // Output the result if successful
  } catch (error) {
    console.error(error.message); // Handle and output any errors
  }
}
// Call the async function to execute
executeAsyncFunction();
```

Este ejemplo muestra cómo async y await pueden simplificar la programación asíncrona en JavaScript. La función asíncrona `myAsyncFunction` simula una operación condicional, devolviendo un mensaje de éxito si se cumple la condición y lanzando un error de lo contrario. La `executeAsyncFunction` utiliza await para llamar a `myAsyncFunction` y maneja el resultado o cualquier error utilizando try y catch. Este enfoque hace que el código sea más fácil de leer y entender en comparación con el manejo de promesas con cadenas de `.then()` y `.catch()`.

3. Solicitudes de Axios

Axios es un cliente HTTP basado en promesas para el navegador y Node.js. Facilita el envío de solicitudes HTTP asíncronas a puntos finales REST y la realización de operaciones CRUD. Axios transforma automáticamente los datos JSON y proporciona una API limpia y sencilla.



3.1 Cómo se utiliza Axios

1. Puedes instalar Axios usando el siguiente comando:

```
npm install axios
```

2. Realizando una solicitud GET

- Axios simplifica la realización de solicitudes HTTP GET. Aquí tienes cómo obtener datos de una API pública:

```
// Import the axios library
const axios = require('axios');
// Using the axios.get method to make a GET request to the specified URL.
axios.get('https://api.example.com/data')
  // If the request is successful, the `.then` block is executed.
  .then(response => {
```

```
    // The response object contains the data returned from the server.
    // We log the `data` property of the response to the console.
    console.log(response.data);
  })
  // If there is an error during the request, the `.catch` block is executed.
  .catch(error => {

    // We log an error message to the console along with the error object.
    // This helps in debugging and understanding what went wrong with the request.

    console.error('Error fetching data:', error);
  });
```

- El método axios.get devuelve una Promesa que se resuelve con el objeto de respuesta, lo que te permite acceder a los datos con response.data.

3. Haciendo una Solicitud POST:

- Para enviar datos a un servidor, utiliza una solicitud POST:

```
// Import the axios library.
const axios = require('axios');
// Data to be sent in the POST request. This is a JavaScript object containing the user information.
const data = {
  name: 'John Doe',
  age: 30
};
// Using the axios.post method to make a POST request to the specified URL with the data object.
axios.post('https://api.example.com/users', data)

// If the request is successful, the `.then` block is executed.
  .then(response => {

// The response object contains the data returned from the server.
// We log a message along with the `data` property of the response to the console.

    console.log('User created:', response.data);
  })
  // If there is an error during the request, the `.catch` block is executed.

    .catch(error => {
    // We log an error message to the console along with the error object.
    // This helps in debugging and understanding what went wrong with the request.

    console.error('Error creating user:', error);
  });
```

- Este fragmento de código demuestra el uso básico de axios para realizar solicitudes HTTP POST y manejar respuestas y errores.

4. **Ejemplo: Usando Async/Await con Axios**

Combinar async/await con Axios proporciona un enfoque limpio para manejar solicitudes HTTP:

```
const axios = require('axios'); // For Node.js, or include via CDN for browser
// Asynchronous function to post data to an API
async function postData() {
  try {
    // Await the response from the Axios POST request
    let response = await axios.post('https://jsonplaceholder.typicode.com/posts', {
      title: 'foo', // The title of the post
      body: 'bar',  // The body/content of the post
      userId: 1     // The user ID associated with the post
    });
    // Log the response data to the console
    console.log(response.data);
  } catch (error) {
    // If there is an error, log the error message to the console
    console.error('Error posting data:', error);
  }
}
// Call the async function to execute the request
postData();
```

Aquí, await pausa la ejecución de la función hasta que se complete la solicitud POST, y el resultado se maneja dentro del bloque try.

Resumen

En esta lectura, aprendiste sobre los conceptos fundamentales de la programación asíncrona en Node.js y Express, centrándote en Promesas, la sintaxis async/await y el cliente HTTP Axios. Las Promesas proporcionan una forma de manejar operaciones asíncronas, haciendo que el código sea más legible y más fácil de gestionar al permitir la encadenación de operaciones. La sintaxis async/await se basa en las Promesas, permitiéndote escribir código asíncrono que se asemeja al código sincrónico, simplificando así el manejo de tareas asíncronas. Finalmente, exploraste cómo usar Axios para realizar solicitudes HTTP, lo cual es esencial para interactuar con APIs y servicios externos. Al dominar estos conceptos, puedes escribir código asíncrono más eficiente y mantenible, lo cual es crucial para construir aplicaciones web robustas.

Autor(es)

Rajashree Patil
Sapthashree K S

